

Structuri de Date și Algoritmi (SDA)

As.Drd.Ing. Bianca Gușită
S1

AC C/TI, AN II

Organizare



- Toate materialele se găsesc pe Campus Virtual
- Mediul de lucru folosit va fi CLion + gitlab.upt.ro. Aplicațiile se vor realiza în limbajul C.
- Fiecare lucrare de laborator va fi citită în avans, de pe site-ul laboratorului.
- **Copierea soluțiilor din alte surse (plagiatul) constituie o încălcare a regulamentului UPT și se va sancționa** conform Regulamentului UPT pentru examinare și notare.
- Dacă aveți întrebări, vă rog să îmi scrieți mesaj pe **Campus Virtual** sau pe **bianca.gusita@upt.ro**.

Organizare



- Nota finală a laboratorului de SDA valorează **50% din media finală a materiei** și va fi media a:
 - 3 note acordate în timpul ședințelor de laborator pe aplicații (Teste de tip cod) **65%**
 - Sortari + căutări tablouri S6
 - Liste S11
 - Recursivitate S13
 - 4 note primite pe teste de tip quiz (notele obținute la testele de tip quiz NU se măresc!) **35%**
 - Test analiza algoritmilor + Scheme logice și tipuri de date S3
 - Sortări (simple + avansate) S7
 - Căutări (șiruri + tablouri) S9
 - Liste (liste pointeri, stiva, coada, multiliste) S11
- Până la sfârșitul săptămânii de recuperare, studentul trebuie să efectueze **toate lucrările de laborator** prevăzute la această materie, conform regulamentului UPT. Se pot recupera 2 laboratoare.

Aplicațiile zilei



- Să se parcurgă **“Tutorial Clion”**;
- Să se creeze un proiect cu o aplicație de tip consolă urmărind pașii din tutorial;
- Să se ruleze o aplicație *”Care calculează suma primelor 5 numere naturale”* urmărind pașii pentru compilarea și rularea unei aplicații în CLion;
- Să se parcurgă **“Tutorial GIT din Clion”**;
- Să se creeze un proiect test pe gilab.upt.ro în care să se încarce un programul realizat anterior.
- Să ne obișnuim cu Debugging
 - Breakpoint
 - Watch pe variabile
 - Step in/step out

Învățăm/Ne reamintim împreună



- **Debugging**

- Procesul de identificare și corectare a erorilor (bugs) dintr-un program.
- Te ajută să găsești și să repari problemele din cod printr-o serie de instrumente care îți permit să:
 - Rulezi codul pas cu pas (step by step) pentru a vedea exact cum se comportă.
 - Pui breakpoints – locuri în cod unde execuția se va opri automat, astfel încât să poți inspecta variabilele și starea programului în acel moment.
 - Verifici valorile variabilelor în timpul execuției pentru a vedea dacă primesc valorile corecte.
 - Monitorizezi fluxul execuției pentru a înțelege mai bine cum se desfășoară programul și unde apare problema.
- **Concluzii:** Acest proces te ajută să identifici erori de logică, probleme cu alocarea memoriei sau alte bug-uri care ar putea duce la comportamente neașteptate ale aplicației.

Învățăm/Ne reamintim împreună

Exemplu



```
#include <stdio.h>

int main() {
    int sum = 0;
    for (int i = 1; i <= 5; i++) {
        sum += i;
    }
    printf("Suma primelor 5 numere este: %d\n", sum);
    return 0;
}
```

Învățăm/Ne reamintim împreună



1. Pune un breakpoint

- Click în marginea stângă pe linia `sum += i;` (sau `for (...)`).
Shortcut: **Ctrl+F8** (Win/Linux) / **⌘F8** (macOS).

2. Pornește debuggerul

- Click pe iconița cu „gândăcel” sau **Shift+F9**. Programul se va opri la breakpoint.

```
#include <stdio.h>

int main() {
    int sum = 0;
    for (int i = 1; i <= 5; i++) {
        sum += i;
    }
    printf("Suma primelor 5 numere este: %d\n", sum);
    return 0;
}
```

Învățăm/Ne reamintim împreună



3. Inspectează variabilele

- În panoul *Debug* vezi *i* și *sum* la *Variables*.
 - Poți adăuga *Watches* (click dreapta → *Add to Watches*).
 - Pentru verificări rapide:
 - *Evaluate Expression* (**Alt+F8** / **⇧F8**) și scrii *sum + i* etc.

```
#include <stdio.h>

int main() {
    int sum = 0;
    for (int i = 1; i <= 5; i++) {
        sum += i;
    }
    printf("Suma primelor 5 numere este: %d\n", sum);
    return 0;
}
```

Învățăm/Ne reamintim împreună



4. Rulează pas cu pas

- **Step Over (F8)** ca să execuți linia curentă fără să intri în funcții.
- **Step Into (F7)** dacă vrei să intri în printf.
- **Step Out (Shift+F8)** ca să ieși din funcția curentă.
- **Resume (F9)** ca să continui până la următorul breakpoint.

```
#include <stdio.h>

int main() {
    int sum = 0;
    for (int i = 1; i <= 5; i++) {
        sum += i;
    }
    printf("Suma primelor 5 numere este: %d\n", sum);
    return 0;
}
```

Învățăm/Ne reamintim împreună



Ce ar trebui să vezi la fiecare iterație?

Iterația	înainte: (i, sum)	după sum += i	după i++
1	(1, 0)	sum = 1	i = 2
2	(2, 1)	sum = 3	i = 3
3	(3, 3)	sum = 6	i = 4
4	(4, 6)	sum = 10	i = 5
5	(5,10)	sum = 15	i = 6 (iese din for)

```
#include <stdio.h>
```

```
int main() {
```

```
    int sum = 0;
```

```
    for (int i = 1; i <= 5; i++) {
```

```
        sum += i;
```

```
    }
```

```
    printf("Suma primelor 5 numere este: %d\n", sum);
```

```
    return 0;
```

```
}
```

- Apoi, **F8** pe printf va afișa 15.

Tip: poți adăuga sum și i la **Watches** sau folosi **Alt+F8 / ⇧F8 (Evaluate Expression)** ca să verifici rapid expresii.

Învățăm/Ne reamintim împreună



Trucuri utile

- **Breakpoint condițional:** click dreapta pe breakpoint → *Condition* → `i == 3` (se oprește doar la iterația 3).
- **Manager de breakpoints:** **Ctrl+Shift+F8** – activezi/dezactivezi rapid.
- Dacă rezultatul nu e 15, verifică: inițializarea `sum = 0`;; limitele buclei (`i <= 5`) și tipurile variabilelor.

```
#include <stdio.h>

int main() {
    int sum = 0;
    for (int i = 1; i <= 5; i++) {
        sum += i;
    }
    printf("Suma primelor 5 numere este: %d\n", sum);
    return 0;
}
```

Aplicațiile zilei



- Să se parcurgă **“Tutorial Clion”**;
- Să se creeze un proiect cu o aplicație de tip consolă urmărind pașii din tutorial;
- Să se ruleze o aplicație *”Care calculează suma primelor 5 numere naturale”* urmărind pașii pentru compilarea și rularea unei aplicații în CLion;
- Să se parcurgă **“Tutorial GIT din Clion”**;
- Să se creeze un proiect test pe gilab.upt.ro în care să se încarce un programul realizat anterior.
- Să ne obișnuim cu Debugging
 - Breakpoint
 - Watch pe variabile
 - Step in/step out